

第59回技能五輪全国大会
「電子機器組立て」職種

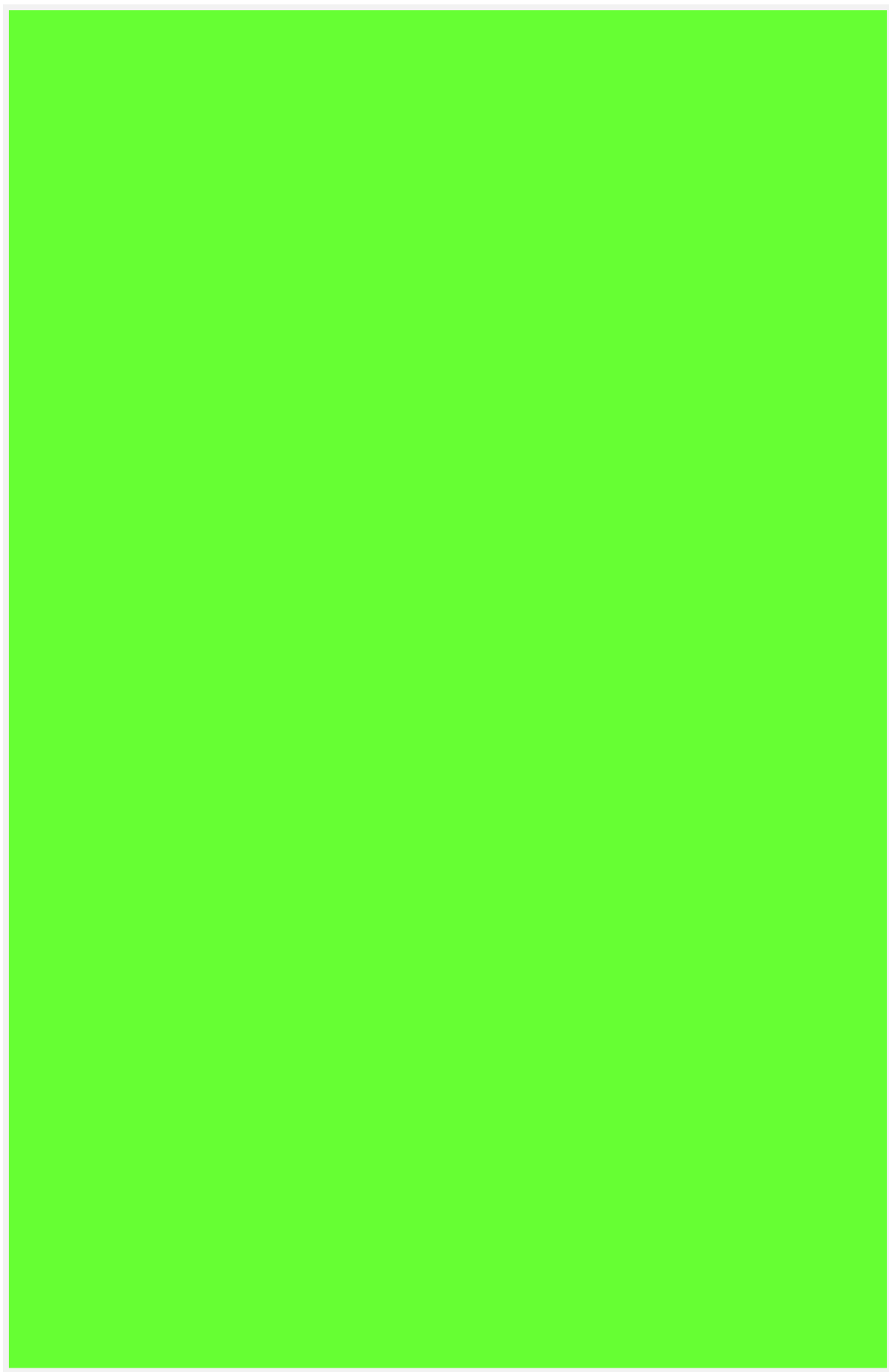
競技Ⅲ

課題仕様書

2021年12月17日
競技時間 2時間

選手番号：

選手氏名：



1.2 MQTT プロトコル

今回使用する ESP32 マイコンは Wi-Fi を内蔵しており，ESP32 が起動すると Wi-Fi ルータに自動的に接続されます．IP は DHCP で振り分けられますが，IP で機器を判別するわけではありません．特殊なプロトコル「MQTT」によって区別しています．このプロトコルは IoT でよく用いられており，今回の課題もそれを用いています．

「MQTT」は 1999 年に IBM 社と Eurotech 社により考案されたプロトコルです．MQTT は Pub/Sub 型通信と呼ばれる種類の通信モデルになります．

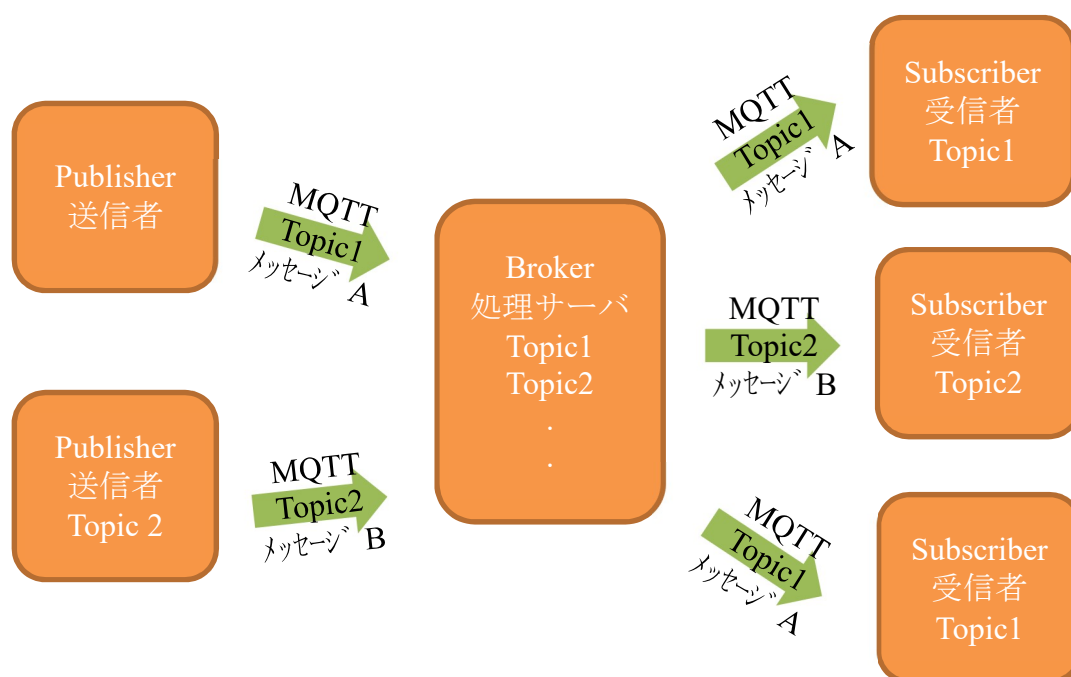


図 1.2 MQTT のしくみ

Publisher（送信者）、Broker（処理サーバ）、Subscriber（受信者）の三つの役割があります．Publisher はデータを Broker に送ります．Broker はそれを処理してデータを Subscriber に送信します．そのデータは複数の Subscriber が受信できます．例えば，温度データを Publisher が送る場合の“実際の温度データ”がメッセージにあたります．

図 1.2 のように Publisher が話題（Topic）を設定し，メッセージが Broker に送られます．Broker は，その話題（Topic）を設定しているすべての Subscriber（購読という意味もある）に届くメッセージを送ります．この方法を用いることで MQTT プロトコルでは，一対多の通信を行うことが可能です．MQTT プロトコルは，Topic とメッセージで構成されています．メッセージは数値や文字，JSON などが扱えます．

Topic は図 1.3 のように “/” で区切られた階層化されたデータをもっています。図では各選手ごとの “Topic” を設定しています。

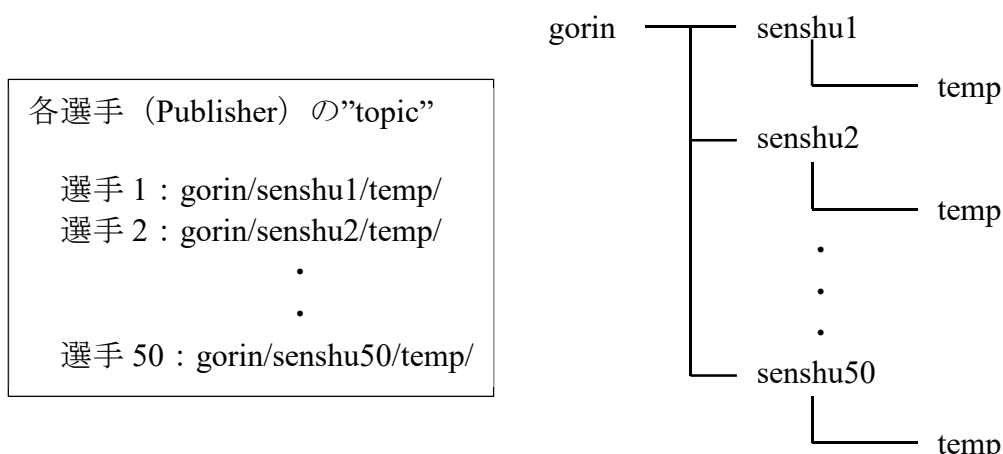


図 1.3 Topic の階層構造

“gorin/senshu1/temp” という Topic では，“gorin” の階層の下に “senshu1” という選手の層があり，さらに各選手には “temp” 層があります。こういう階層構造は PC でいうところのフォルダの PATH のようなものを思い浮かべると分かりやすいかもしれません。二つ目の階層は，“senshu2” や “senshu50” など選手階層で，三つ目の階層である “temp” は，全選手同じ名前の階層です。

図 1.3 に示すように Subscriber が選手 1 の温度データを取得したい場合は，“gorin/senshu1/temp/” と設定するとデータを取得することができます。また全選手の温度を集めたい場合は “gorin/+temp” と Topic を設定すれば獲得できます。“+” はワイルドカードであり，“その階層すべて”の意味に相当し，また下の階層の “temp” が同じ名前であることで，全選手の温度データという意味になります。大量の機器やセンサのデータを管理し区別できる IoT に適したプロトコルです。

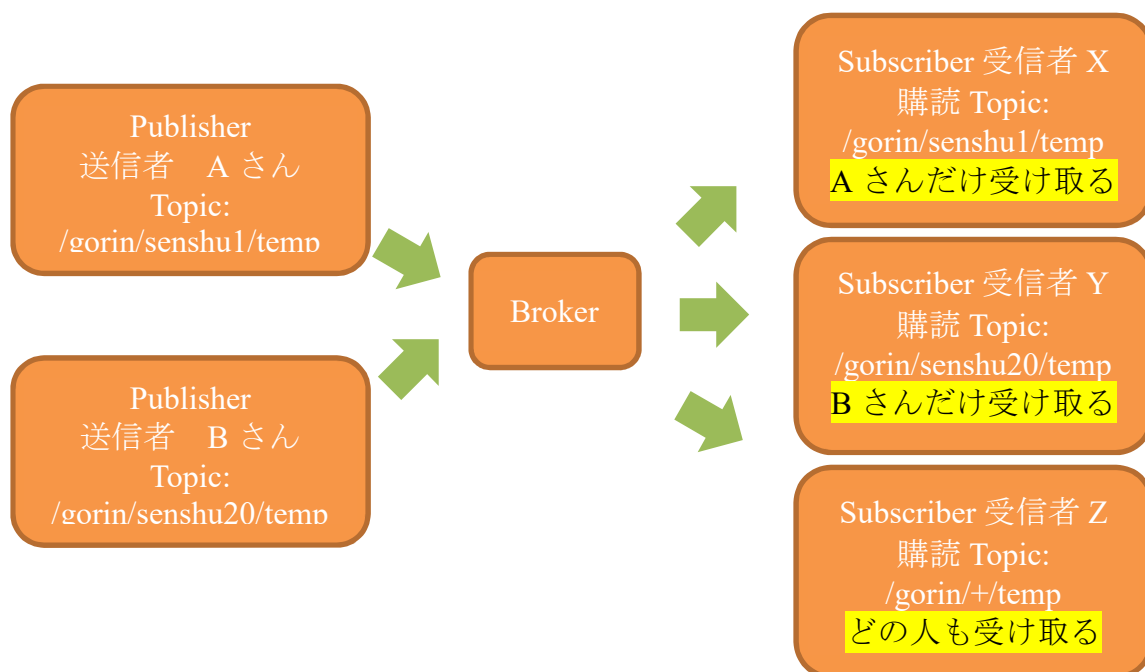


図 1.3 Topic の考え方

MQTT プロトコルを用いることにより，大量の機器からの情報から自分の欲しいデータをフィルタリングし収集，解析することが可能です．今回は，ネットワークや MQTT についてあまり気にしなくてもよい課題になっています．

1.3 機器の構成

(1) 構成機器

プログラム設計課題で使用する電子機器は，「IoT ボード」です．図 1.4 に機器のブロック図を示します．IoT ボードの構成機器は，以下のとおりです．

- IoT ボード
- マルチファンクションボード（選手持込み）
- カップリングボード（選手持込み）
- AC アダプタ（DC 9V, 1.3 A）（選手持込み）

専用基板である「IoT ボード」は，マルチファンクションボードとカップリングボードを介して接続します．

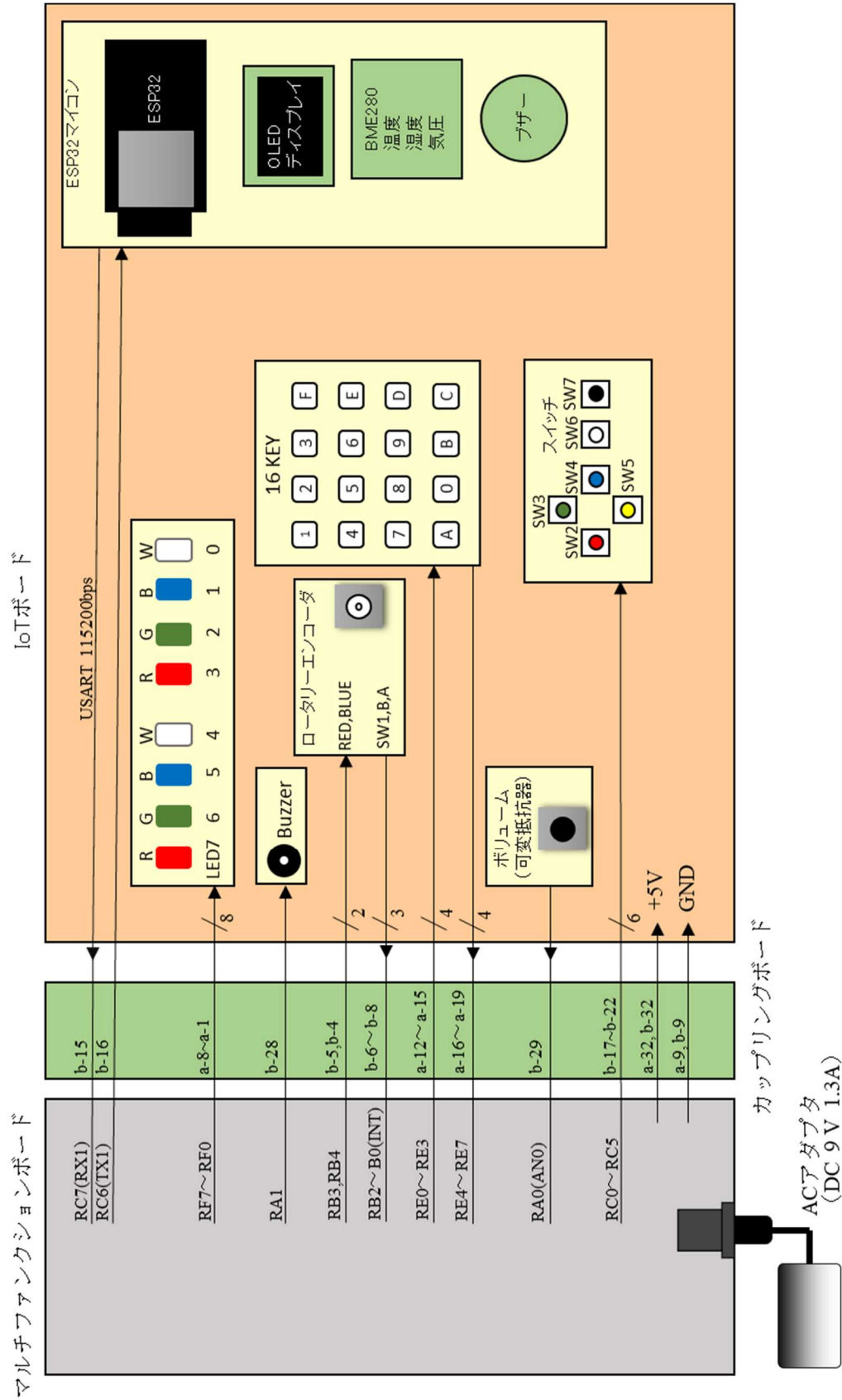


図 1.4 IoTボードのブロック図

(2) IoT ボードの動作概要

「IoT ボード」は、カップリングボードを介し、マルチファンクションボードと接続します。IoT ボードには、ESP32 マイコンが搭載されており、Wi-Fi 接続、センサ (BME280)、OLED ディスプレイ、ブザーの制御に使用しています。それ以外の LED0～7、ブザー、ロータリエンコーダ、可変抵抗器、スイッチ SW2～SW7、16 キーパッドはマルチファンクションボードから直接制御を行います。

ESP32 マイコンは、Wi-Fi 接続をおこない、ネットワークにアクセスができます。ネットワーク上で、MQTT を用い Publisher, Subscriber となりデータのやり取りを行います。また、センサ (BME280) では温度、湿度、気圧が測定できます。

OLED ディスプレイは、グラフィックディスプレイで 128×64 ドットの有機 E L ディスプレイで文字が 1 行に 13 文字 4 行表示することができます。

マルチファンクションボードの PIC とは USART で接続していますが、MQTT やセンサ、ディスプレイは専用の関数でやり取りを行うため、USART を意識する必要はありません。

LED0～7 は、LED7 と LED3 が赤 (R)、LED6 と LED2 が緑 (G)、LED5 と LED1 が青 (B)、LED4 と LED0 が白 (W) になっています。ブザーは、HIGH になると鳴ります。ロータリエンコーダは、赤と青の LED が搭載され、また押しボタンスイッチ SW1 と A 相 B 相があります。これはモードの切り替えなどに使用します。この部品は選手がプログラムで扱うことはありません。16 キーパッドは、マトリックス状にスイッチが配置されており、4 列の入力 X1～X4、4 行の出力 Y1～Y4 が接続されています。

ボリューム (可変抵抗器) は、0 V から 5 V までの電圧を出力し、アナログ入力 AN0 に入力されます。

スイッチは、SW2～SW7 まであり、某ゲームコントローラ的な配置になっており、それぞれ色が決まっています。

競技Ⅲでは、IoT ボードを用いたプログラミング課題とした競技を実施します。

(3) IoT ボードの動作チェックと測定課題

サーバ内にある“`iot_check.hex`”をダウンロードし書き込むことで IoT ボードのチェックができます。ESP32 マイコンはそのまま構いませんが、電源をオンにしてから“ピピッ”とブザーが鳴って Wi-Fi に接続してから使用して下さい。

各部品のチェック方法を表 1.1 に示します。

表 1.1 IoT ボードの動作チェック

部品	方法	動作
ロータリエンコーダ	回転	OLED ディスプレイの 1 行目に現在のモードが表示されています モードの切り替え (mode1～mode7 がループ) 時計回り → mode が進む 時計回り → mode が戻る

SW1	押下	ロータリエンコーダの LED 押下前 青 → 押下後 赤 押下している間 ブザーが鳴ります
SW2 (赤) SW3 (緑) SW4 (青) SW6 (白)	押下	押下したスイッチと同色の 2 つの LED が点灯 SW2 赤→ LED7, LED3 点灯 SW3 緑→ LED6, LED2 点灯 SW4 青→ LED5, LED1 点灯 SW6 白→ LED4, LED0 点灯
SW5 (黄)	押下	MQTT 通信 SW5 押下でサーバに送信 OLED ディスプレイの 2 行目に他の表示が消え “MQTT_OUT” と表示し, サーバとの通信成功時に OLED ディスプレイの 3 行目に “ok” が 2 秒程度表示
SW7 (黒)	押下	16 キーパッドの入力終了に使用
ボリューム	回転	OLED に値が表示 0~255
16 キーパッド	押下	いずれかのキーを押下すると OLED ディスプレイの表示が切り替わり入力モードになります. 入力したキーを OLED ディスプレイに表示し, SW7 で入力が終了になり, 通常のチェックモードになります.
BME280		最初から OLED の 2 行目に温度 (華氏) が表示されています.

2 プログラム設計課題

2.1 課題内容

プログラム設計競技の主な課題は、6 つです。IoT ボードとマルチファンクションボードを接続して、ボード上の部品を制御します。デジタル IO 制御、A/D 変換、OLED ディスプレイ制御、センサデータ処理、IoT 通信制御の内容を含み、仕様どおりに制御するプログラムを設計、記述、実装を行ってください。

プログラム設計は、**公表 2**『3-5 プログラム設計競技仕様』、および以下に示す『プログラム設計仕様』に基づいて行ってください。また、ソースプログラムの記述方法は、**公表 2**『4-4 コーディング作法のガイドライン』に従ってください。

(1) IoT ボード

図 2.1 に示す IoT ボードは、LED、ブザー、ロータリスイッチ、タクトスイッチ、ボリューム、16 キーパッドおよび、ESP32 マイコンとその周辺機器で構成されています。IoT ボードは、カップリングボードを介してマルチファンクションボードに接続します。また、配布されたマルチファンクションボード用のスペーサを取り付けてください。

IoT ボードの電源を ON にすると、ESP32 上の LED が赤く点灯します。その後 ESP32 が Wi-Fi に接続されると、ブザーが「ピピッ」と鳴ります。鳴らない場合は、もう一度電源を入れ直して下さい。Wi-Fi に接続できない場合、センサ、OLED ディスプレイ、MQTT 通信は動作しません。

2.2 プログラム設計仕様

(1) 配布したソースファイルの構成

配布したソースファイルは，“iot_main.c”，“iot_xx.c”，および“iot_xx.h”の三つです。ソースファイルの構成を図 2.1 に示します。iot_main.c には，main 関数や各種初期設定，タイマ割込み処理が記述されています。iot_xx.h には，define マクロや課題のプロトタイプ宣言が記述されています。iot_xx.c には，課題の処理関数が用意されていますので，各課題の仕様にしたがった処理関数のプログラムを記述してください。ただし，課題の処理関数以外の場所，例えばグローバル変数や新たな関数，割込み処理などを，iot_xx.c，iot_xx.h，iot_main.c 中に記述してもかまいません。

- iot_main.c: main()関数が入っているファイル（提出）
- iot_xx.c : 課題の処理関数が入っているファイル（提出）
- iot_xx.h : ヘッダファイル（提出）

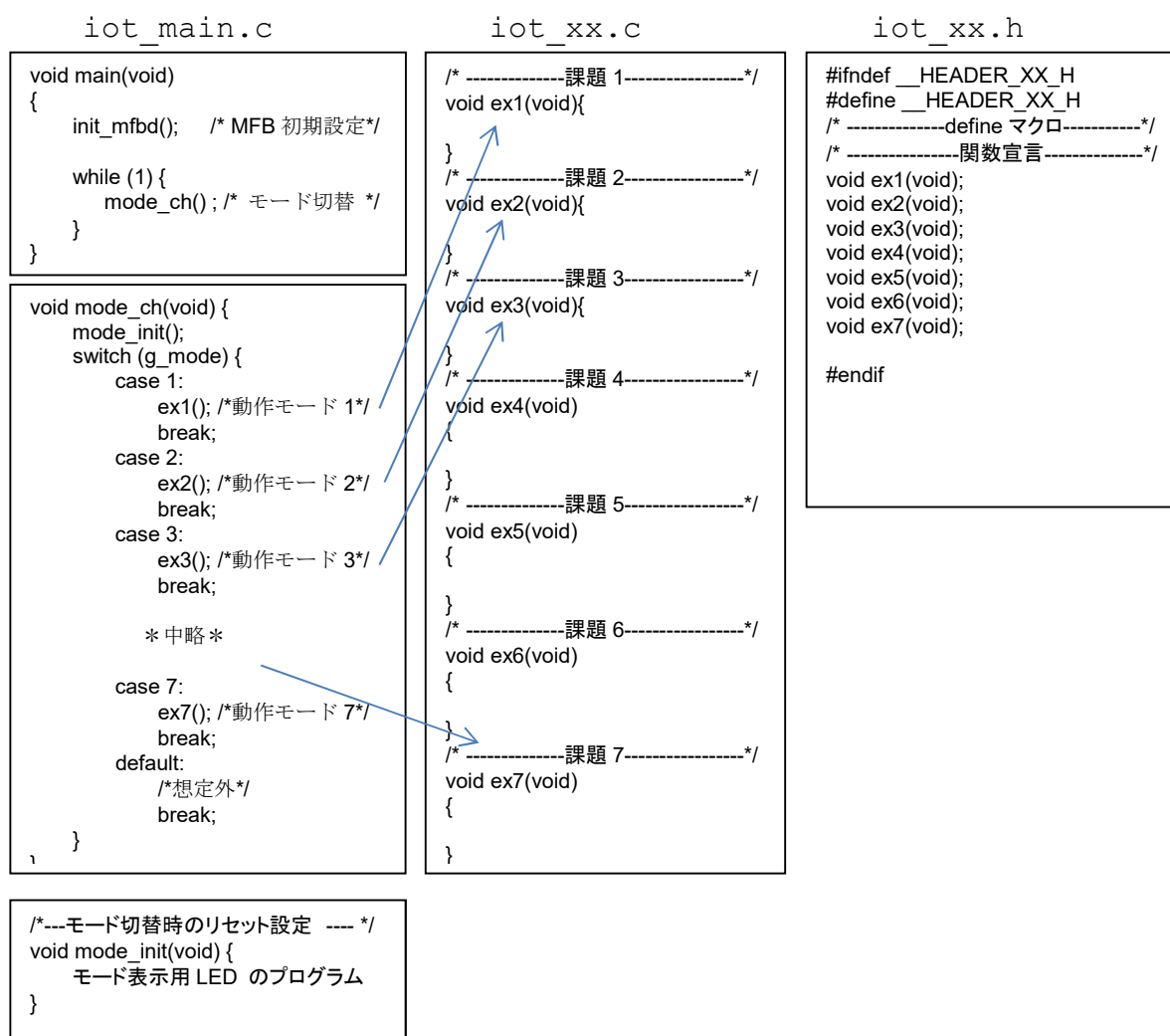


図 2.1 配布したソースファイルの構成

(2) 動作モード

以下の6個の動作モードのプログラムを設計、作成、実装することが課題です。

- モード1：スイッチ操作によるLEDの点滅
- モード2：流れるLEDの点灯
- モード3：LEDの明るさを調整
- モード4：華氏から摂氏の変換
- モード5：IoT的問題 データの送受信
- モード6：16 キーパッドの入力プログラム

動作モードの切り替えは、ロータリスイッチで行います。時計周りでモードの番号が進み、反時計回りで番号が戻ります。モード6の次はモード1に戻り、モード1を戻すとモード6になります。OLEDディスプレイの1行目に現在のモード番号が表示されます。モード切替に関するプログラムは変更しないでください。

- 電源投入時は、ESP32がネットワークに接続されて“ピッ”と鳴ります。
- ネットワークロータリスイッチのLEDが電源投入時のネットワーク接続成功時のみ青くなりますが、その後ロータリスイッチの色が変わる場合がありますが問題はありません。
- 電源投入時は、モード1のex1()が実行されます。
- モード1からモード6までのプログラムは、それぞれの処理関数ex1()～ex6()に記述してください。ただし、タイマ1やその設定に関する部分などは、処理関数以外のところに記述してもかまいません。
- 処理関数ex1()～ex6()の仮引数と戻り値は、voidです。

(3) グローバル変数

表2.1に定義されている使用可能なグローバル変数を示します。これら以外のグローバル変数（システムグローバル変数）は、あらかじめ関数に使用している変数であるため使用しないでください。グローバル変数とローカル変数を区別するために、変数名の頭文字に“g_”を付けています。

表 2.1 定義されているグローバル変数

グローバル変数	設定内容	割込み
g_mode	現在のモード用	—
g_flag_t0	タイマ0 割込みフラグ (10 ms 秒)	タイマ0 割込み
g_flag_t1	タイマ1 割込みフラグ	タイマ1 割込み
g_flag_delay10ms	10 ms 単位のディレイ関数用フラグ	タイマ0 割込み
return_mqtt_flag	MQTT データ取得用フラグ	
return_mqtt	サーバから返ってきたデータ	

- g_mode は、モード番号を格納している変数です。
- g_flag_t0 は、タイマ0 割込みで割込みが実行されるごとに、「0」と「1」を交互に切り替えるトグル動作になっています。
- g_flag_t1 は、タイマ1 割込みで割込みが実行されるごとに、「0」と「1」を交互に切り替えるトグル動作になっています。
- g_flag_delay10ms は、10ms のディレイ関数のフラグです。

- `return_mqtt_flag` は、課題 5 で使用します。サーバからデータが返ってくるとフラグが“1”になります。
- `return_mqtt` は、課題 5 で使用します。サーバから返ってきたデータが格納されます。

(4) 初期設定

使用するポートの設定は、あらかじめ図 1.4 に示す入出力装置, A/D 変換, USART の構成に適合するように設定してあります。また、タイマ 0 は 10 ms のインターバルタイマとしてディレイ関数に使用していますのでタイマの値は変えないでください。タイマ 1 については初期設定を行い、割込み処理の場所を設けてありますが、ユーザ用に設けていますので変更して使用してもかまいません。

(5) 関数

ディレイ用の関数や、課題に使用する関数を用意してあります。必要に応じて使ってください。

表 2.2 定義されている関数

関数名	内容	使用例
<code>delay_10ms(unsigned int)</code>	10ms 単位のディレイ 引数×10ms=ディレイ時間	1 秒ディレイ 例 <code>delay_10ms(100)</code>
<code>void wait_1ms(unsigned int value)</code>	1ms ディレイ 引数×1ms=ディレイ時間	10ms ディレイ 例 <code>wait_1ms(10)</code>
<code>void req_OLED_clear(void)</code>	OLED ディスプレイの画面クリア OLED ディスプレイは、クリアしない限り画面はそのままです。	一般的な使い方は ①画面のクリア ②表示したいデータをバッファにおくる ③実際に表示 例 画面の 2 行目に“TEST”を表示する <code>req_OLED_clear();</code> <code>req_OLED_buf("TEST",0,2)</code> <code>req_OLED_display();</code>
<code>int req_OLED_buf(char *string_data, int *col, int *row)</code>	OLED ディスプレイのバッファ アルファベット、数字、記号はほぼ表示できます。 表示を変更したい場合は、まずバッファにおくってください。 引数①: <code>*string_data</code> (char 型) 表示したいデータ 引数②: <code>*col</code> (int 型) 表示したい列の番号 最大 13 列 (1~13) 引数③: <code>*row</code> (int 型) 表示した行の番号 最大 4 行 (1~4)	
<code>void req_OLED_display(void)</code>	OLED ディスプレイの表示 バッファにたまったデータを表示します	
<code>int read_VR(void)</code>	ボリュームの情報取得 (A/D 変化値) 0~255	<code>Val= read_VR();</code>

float req_BME280(char *sensor);	センサ情報の取得 引数 char *sensor temp : 温度取得 (華氏) 戻り値 (float 型) 小数点 2 桁で返ってきます	温度を取得したいとき t = req_BME280("temp")
void publish_MQTT(char *topic, char *payload)	MQTT プロトコルのパブリッシュ (送信) 引数① : *topic (char 型) トピック 文字列 引数② : *payload (char 型) メッセージ 文字列	MQTT プロトコルでの送信 トピックを "/kadai4" に設定し 値 "a=7" をパブリッシュする publish_MQTT("/kadai4","a=7")

2.3 プログラム設計課題

6つの課題があります。それぞれの課題では、提示した仕様以外の動作については問いません。

(1) 課題1：モード1 ex1(void)関数

押下したスイッチの色と同色のLEDが点滅する。

(仕様)

- 表2.3に示すように押下したスイッチと点滅するLEDが対応していること。
- スイッチを押下している間だけLEDが点滅すること。
- 目で見えてわかる程度の速さで点滅すること。
- 同色のLEDはすべて点滅すること。
- 押下した瞬間の状態は、点灯、消灯のどちらでもよい。

表 2.3 課題1 スイッチとの対応 LED

スイッチ	LED
SW2 (赤)	LED7, LED3
SW3 (緑)	LED6, LED2
SW4 (青)	LED5, LED1
SW6 (白)	LED4, LED0

(2) 課題2：モード2 ex2(void)関数

スイッチ7を押して離れたときから、8個のLEDの点灯が左から右へ移動を繰り返す。また、スイッチ6で消灯する。

(仕様)

- LED0～LED7を点灯範囲とすること。
- 初期状態は、すべてのLEDが消灯していること。
- 初期状態でスイッチを押して離れたとき、LED7から点灯が移動すること。
- 目で見えてわかる程度の速度で点灯すること。
- 移動している際、SW7を押下すると移動が停止し、離すとその位置から移動が継続すること。
- 左から右への点灯を繰り返すこと。
- スイッチ6を押下するとLEDがすべて消灯し、初期状態に戻ること。

(3) 課題3：モード3 ex3(void)関数

ボリュームを変えると8個のLEDの明るさが変化する。

(仕様)

- アナログ的に明るさが変化する。
- LED0～LED7の明るさが同時に変化する。
- ボリュームを時計回りで回転すると明るくなる。
- ボリュームを反時計回りに回転すると暗くなる。

- どのような状態でも LED が点滅しているように見えないこと.
- 反時計回りに回し切ると消灯すること.

(4) 課題 4 : モード 4 ex4(void) 関数

温度センサから華氏の温度データを読み込み, 値を変換し OLED ディスプレイに表示する.

- 温度データの更新は, おおよそ 500 ms ごとに行うこと.
- ディスプレイへの表示は, 2 行目に華氏の温度, 3 行目に摂氏の温度を表示すること.
- 小数点以下 2 桁で表示すること.
- それぞれの数値の後に, 華氏は「F」, 摂氏は「C」の単位を付けること.
- 摂氏と華氏は, 1 次関数の式で表される. 表 2.4, 図 2.2 を参考に関数式を考えること.

表 2.4 摂氏と華氏の換算

摂氏 °C	華氏 °F
-5.00	-20.56
0.00	-17.78
10.00	-12.22
23.00	-5.00
32.00	0.00
40.00	4.44
50.00	10.00
55.00	12.778

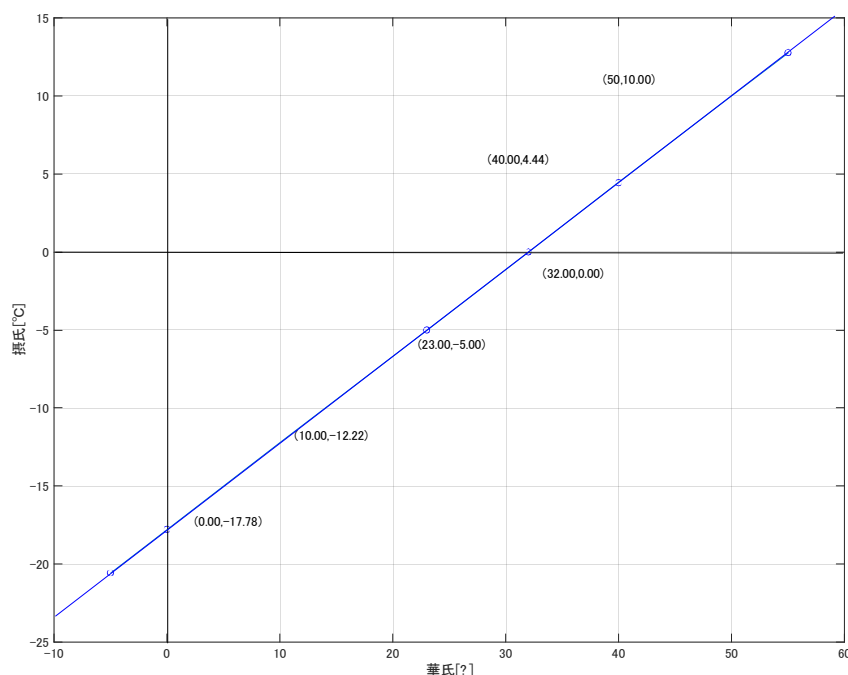


図 2.2 華氏と摂氏の関係

(5) 課題5：モード5 ex5(void)関数

SW6 を押下し，MQTT プロトコルを用いてサーバにアクセスする．サーバから返ってきた4桁の16進数を10進数に変換してOLEDディスプレイに表示するとともに変換されたデータをサーバに送り直す．

- ループで送信し続けた場合，サーバがダウンする恐れがあるため，SW6を押下するごとに，1回だけMQTT送信すること．
- MQTTプロトコル関数のtopicには「/kadai5/data/req」，payloadには「request」を入れ実行し，100ms程度時間待ちを行う．その後return_mqtt_flagが“1”となりOLEDの2行目に4桁の16進数が表示される．また変数“return_mqtt”にサーバからのデータ（テキストデータ）が格納される．
- return_mqtt_flagは，自動で“0”に戻らないため手動で“0”に戻すこと．
- 4桁の16進数を符号なし10進数に変換すること．
- 変換したデータをOLDディスプレイの3行目に表示すること．
- 変換したデータをMQTTプロトコル関数のtopicには「/kadai5/ans」，payloadには「変換した値」（テキストデータ）をいれ実行．
- 変換したデータをサーバに送り，正解の場合OLEDの4行目に“correct”と表示される．

(6) 課題6：モード6 ex6(void)関数

16キーパッドのキーを押下するごとに，キートップの文字をOLEDディスプレイに表示する．

- キーを押すとキートップに書かれた文字をOLEDディスプレイの2行目左端から表示すること．
- 右端（13文字）まで表示された後は，下の行の先頭に移動すること．
- 電卓のようにキーを押下し続けても連続で同じキーが入力されないこと．

<ヒント>

以下にキースキャンの考え方のヒントを示しています．なお，必ずしもこれに従う必要はありません．

- ② 図2.2のようにマトリックス状にキーが配置されている．
- ② 列をX1~X4，行をY1~Y4とし，列が入力ポートに，行が出力ポートに接続されている．
- ③ キー押下することで，それぞれ対応したXとYが接続される
- ④ 出力ポートを制御し，入力をスキャンすることで入力されているマトリックスの位置が特定できる．

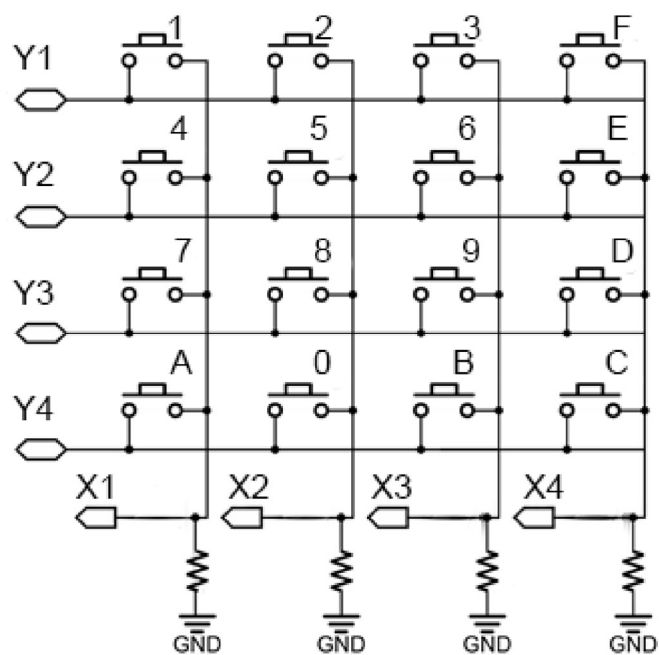


図 2.3 マトリックスキーパッド

3 測定課題

3.1 課題内容

測定競技の課題は、IoT ボード上の ESP32 マイコンから出力されている波形の観測を行い、観測した波形から繰返し周波数，周期を測定することです。

測定は、**公表 2**『3-7 測定競技仕様』、および以下の『測定仕様』に基づいて行いなさい。

3.2 測定仕様

(1) 測定回路

測定は、IoT ボード上で行うこと。

(2) 測定条件

- ESP32 マイコンの IO33 の出力（J2 の 8 番端子 ESP32 上のシルクに印刷されている）の GND に対する電圧波形をオシロスコープで観測すること。
- 観測した波形画像をパソコンに取り込み、サーバで配布した「競技Ⅲ 測定シート」に貼り付けること。
- 観測した波形を読むために必要なオシロスコープの設定条件を「競技Ⅲ 測定シート」に記述すること。
- 観測した電圧波形の繰返し周波数を求め、「競技Ⅲ 測定シート」に記述すること。

4 提出仕様

4.1 提出状態

IoT ボード、およびドキュメント類の提出状態については、**公表 2** 『3-8 提出仕様（全競技に適用）』、および以下の仕様にしてください。

(1) IoT ボード

IoT ボードの提出状態は、表 4.1 に示すとおりとすること。

表 4.1 IoT ボードの提出状態

可変抵抗器 VR1	左に回し切った状態
64 ピンコネクタ	IoT ボードとマルチファンクションボードが、正しく接続された状態
荷札	荷札の針金を、試作基板を正面から見て IoT ボードの左下のスペーサ部のねじと一緒に締め込み、容易に外れないように取り付けた状態

(2) マルチファンクションボード

マルチファンクションボードの提出状態は、表 4.2 に示すとおりとすること。

表 4.2 マルチファンクションボードの提出状態

電源スイッチ SW1	オフ
スライドスイッチ SW3	Writer 側
スライドスイッチ SW4	Bus Line 側
機能拡張用コネクタ	何も挿入されていない状態
DC ジャック	何も接続されていない状態 (電源アダプタは提出しない)
64 ピンコネクタ	IoT ボードとマルチファンクションボードが、正しく接続された状態
SD メモリカードスロット	何も挿入されていない状態
モジュラジャック	何も接続されていない状態
USB コネクタ	何も接続されていない状態
無線通信モジュールコネクタ	何も挿入されていない状態
PIC18F6722 ボード(MPU ボード)	作成したプログラムを書き込んだ状態

(3) プログラムソースリスト

提出するプログラムソースリストは、“iot_xx.c”、“iot_main.c”、およびヘッダファイル“iot_xx.h”です。なお、上記以外のソースリストがある場合は、併せて提出してください。

(4) 提出する「課題作品」、「提出用封筒」、および「配布&回収用封筒」は、あらかじめ配布した用箋はさみの上に置くこと。

5 注意事項

5.1 質問

- (1) 質問があるときは、コールシステムを用いて行いなさい。
- (2) 公表資料（ **公表 1**， **公表 2** ），職種連絡会資料に明確に記述されている事項，配布したドキュメント，資料に明確に記述されている事項，競技開始前に明確に説明した事項，掲示板に掲載されている事項に関する質問には回答できません。
- (3) 持込み機器（コンピュータ，測定器，電子機器など），および搭載ソフトウェア（Altium Designer，MPLAB X IDE，XC8 C コンパイラ，オシロスコープソフトウェア，ターミナルソフトウェアなど）の不具合，使用方法に関する質問には回答できません。
- (4) 質問に対する回答が即座にできない内容に関しては，競技委員間で協議の上，回答します。

5.2 資料，データシート

- (1) 別添資料 3 の「競技Ⅲ 部品表」のデータシート欄に「○」がついている部品，材料などの資料，データシートなどは，サーバ上の“datasheet3.zip”に PDF 形式で保存されています。これらの資料，データシートなどは必要に応じて，使用してください。